OYANI Muzhgan
LACROIX-FARDET Arnaud
Groupe : S5-A1

Projet info0502
Gestionnaire de QCM

Lien du dépôt git : https://gitlab-mi.univ-reims.fr/lacr0040/projet_info0502

Table des matières

Introduction : 2
Architecture du projet : 2
Services et implémentation : 2
Services principaux 2
Base de données 3
Echanges client / serveur : 3
Analyse du projet : 4
Chronogrammes : 4
Echanges principaux : 4
Enregistrement : 5
Authentification : 5
Réalisation d'un QCM : 6
Déconnexion : 6
Représentation des données : 7
Modélisation des classes : 7
Client 7
Serveur 7
Tests : 7
Documentation : 8
Jeu d'essai : 8
Conclusion : 11

Introduction :

Le projet visait à réaliser un gestionnaire de QCM à l'aide d'un échange client / serveur en utilisant des files de messages MQTT. Les choix d'implémentation et les différentes modélisations sont décrits dans la suite de ce rapport.

Architecture du projet :

Services et implémentation :

Services principaux

Le gestionnaire de QCM propose plusieurs services.

Tout d'abord, un utilisateur peut s'enregistrer. Il renseigne son identifiant, unique sur le serveur, son mot de passe, son adresse email, et son rôle, pour savoir s'il est étudiant, professeur, ou s'il a un profile mixte (les rôles seront décrits plus loin). Une fois enregistré, l'utilisateur est directement connecté et reçoit son identifiant de session. Lors de

l'implémentation, aucune sécurité n'a été mise en place pour la gestion du mot de passe : l'utilisateur écrit directement dans la console et le voit clairement, et il n'est pas chiffré ni dans l'envoi des données, ni dans le stockage dans la base de données (voir section dédiée). Du point de vue du serveur, un utilisateur connecté est représenté par un Objet (voir la modélisation des classes).

Alternativement, si un utilisateur était déjà enregistré, il peut directement se connecter. Il devra renseigner son identifiant et son mot de passe, et, s'il n'y a pas d'erreurs, il recevra son identifiant de session.

Un utilisateur authentifié et disposant du rôle STUDENT ou MIX peut réaliser un QCM. Pour cela, il envoie au serveur une requête qui contient l'identifiant du QCM souhaité. Le serveur vérifie d'abord que ce QCM existe dans la base de données, puis renvoie l'ensemble des question et propositions associées à la demande du client.

Le client se charge d'afficher ces questions à l'utilisateur et de collecter ses réponses. Une fois toutes les réponses saisies, elles sont envoyées en une seule fois au serveur, qui procède la correction. Le score obtenu est ensuite renvoyé au client, qui se charge de l'affichage.

Enfin, un utilisateur connecté et ayant le rôle d'étudiant ou mixte, peut demander à voir ses résultats. Il n'a qu'à faire une demande du service et d'envoyer son identifiant de session, et le serveur enverra les données correspondantes. L'affichage est réalisé dans le code du client.

Base de données

Pour stocker les différents QCM et les différents utilisateurs, une base de données a été mise en place. Dans le serveur se trouve deux fichiers textes contenant chacun un objet JSON. Pour chaque ajout, le serveur récupère cet objet, fait les modifications nécessaires, et écrit l'entièreté de l'objet dans le fichier correspondant, remplaçant ainsi les données précédentes.

Echanges client / serveur :

Pour l'échange des données, les différents acteurs utilisent des files de messages MQTT (voir chronogrammes). Chaque message envoyé est un objet JSON :

Pour le client, l'objet suit ce format :

```
{
    "sessionId": "",
    "service": "",
    "params": {}
}
```

La clef « sessionId » correspond à l'identifiant de session envoyé par le serveur lors d'une connexion. « service » correspond au service du serveur demandé par le client. Et enfin, « params » est un objet JSON dans lequel toutes les données nécessaires pour le service demandé sont stockées, par exemple l'identifiant et le mot de passe pour la connexion.

Pour le serveur, l'objet suit ce format :

```
{
    "status": "OK/KO",
    "error": "",
```

```
    "params": {}
}
```

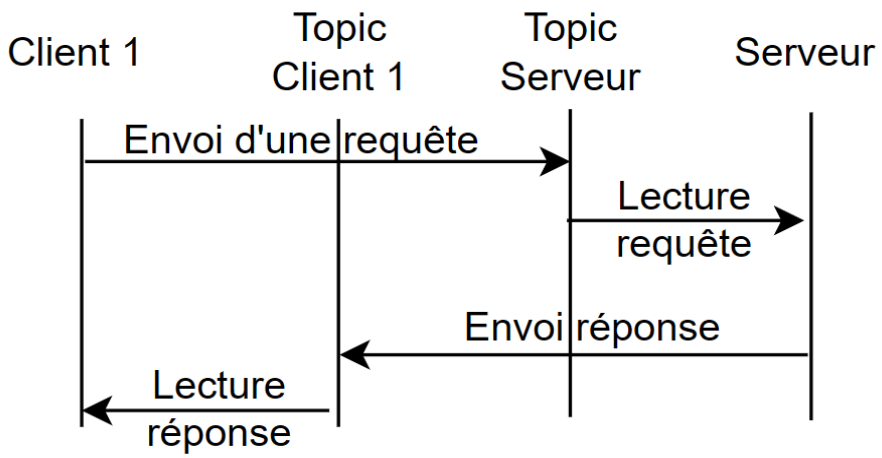
La clef « status » est la chaîne de caractère « OK » si le service effectué n'a pas rencontré d'erreur, et « KO » s'il y a la moindre erreur, quelle soit au niveau des données envoyées ou de l'exécution du service. En lien avec cette clef, « error » contient le message d'erreur pour le client ; la propriété n'est présente que si le « status » est « KO ». Enfin, « params » est identique au client.

Analyse du projet :

Chronogrammes :

Echanges principaux :

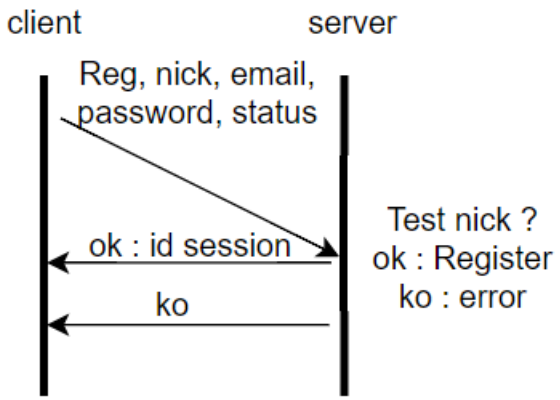
Pour l'ensemble du projet, des files de messages MQTT ont été utilisé. Le serveur reçoit les différentes requêtes des clients sur un seul topic puis envoie une réponse sur un topic spécifique audit client. Ainsi, pour chaque client qu'il pourrait y avoir de connecté au même moment, le serveur enverra un message sur un topic différent.



Etant donné que tous les échanges réalisés pour ce projet on été fait de cette manière, pour simplifier les schémas suivants, il ne sera indiqué que deux acteurs, client et serveur, mais il faut noter que les échanges passent par des files de messages.

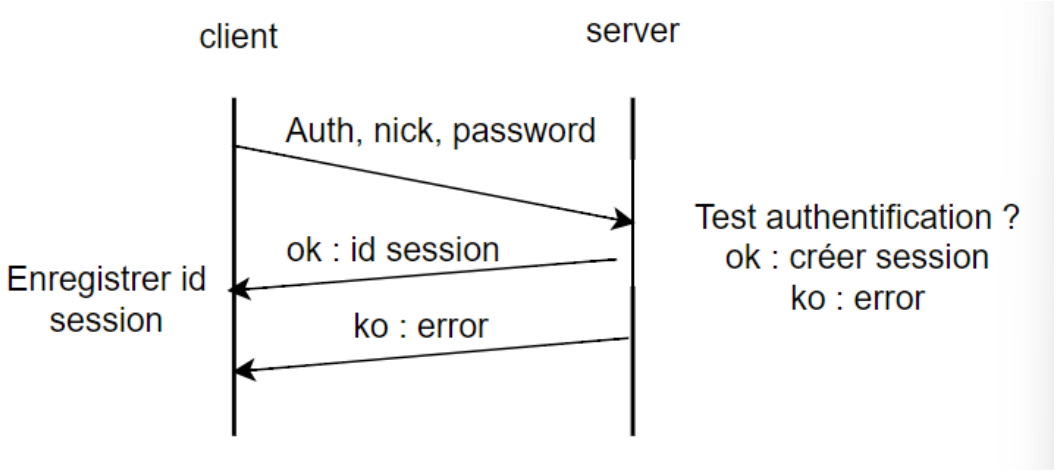
Enregistrement :

Un utilisateur non inscrit peut s'enregistrer et se connecter en envoyant une requête avec son Nick, son Email, son mot de passe et son statut. Le serveur vérifie que le Nick n'existe pas déjà dans la base de données, puis ajoute l'utilisateur dans la base de données. Une fois l'enregistrement terminé, l'utilisateur est automatiquement connecté et reçoit un identifiant de session unique.



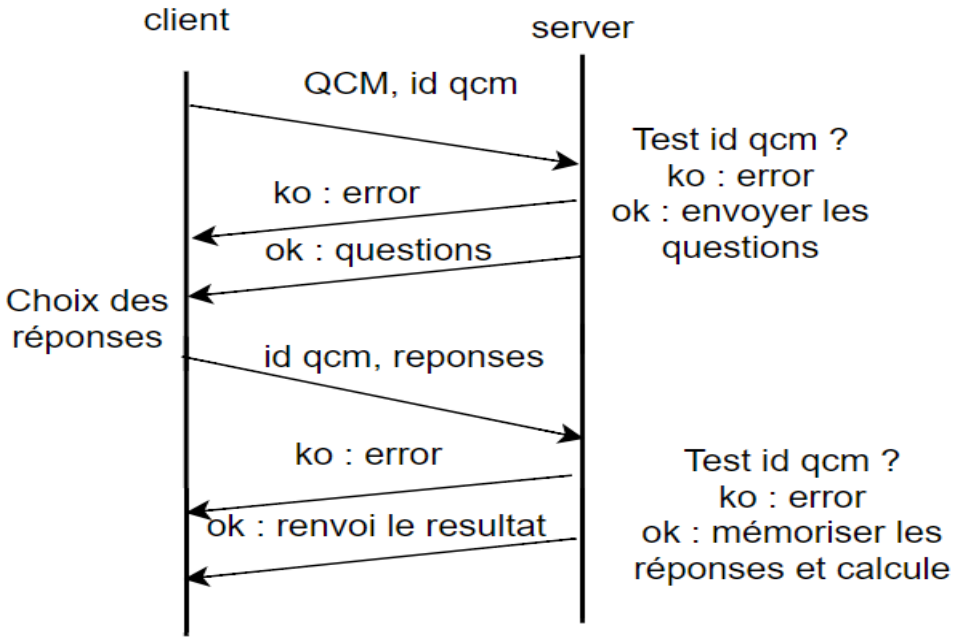
Authentification :

Un utilisateur déjà enregistré peut se connecter en fournissant son Nick et son mot de passe. Le serveur vérifie la correspondance dans la base de données. Si la vérification réussit, il génère un identifiant de session qui sera utilisé pour les requêtes suivantes. Si les informations sont incorrectes, le serveur renvoi un message d'erreur.



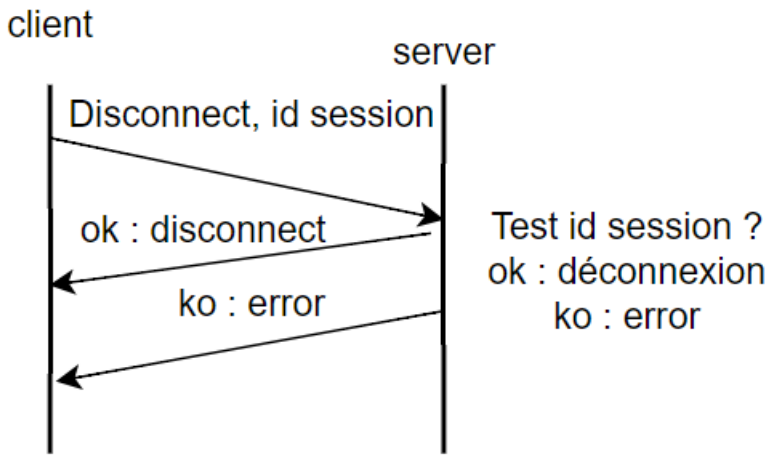
Réalisation d'un QCM :

Un utilisateur connecté et ayant le statut STUDENT ou MIX peut demander à réaliser un QCM. Il envoi au serveur l'identifiant du QCM. Le serveur vérifie si le QCM existe dans la base de données, puis renvoi l'ensemble des questions et ses propositions associées. Le client affiche les questions et recueille les réponses de l'utilisateur, puis renvoi les choix de l'utilisateur. Le serveur corrige le QCM, calcule le score obtenu et renvoi le résultat au client.



Déconnexion :

Il est possible pour un utilisateur connecté de se déconnecter. Le client envoie une requête comportant l'identifiant de la session et le serveur supprime cette session de sa liste d'utilisateurs connectés.



Représentation des données :

La base de données de notre projet repose sur deux fichier JSON qui sont stockés côté serveur. D'abord, le fichier « users.txt » contient chaque utilisateur, représenté par un objet JSON contenant Nickname, Mot de passe, Score, Email et Statut, en respectant le format suivant :

{"Thomas":{"password":"good123","scores":[],"email":"Thomas@mail.com","status":"STUDENT"}

Le deuxième fichier, « mcq.txt », contient l'identifiant d'un QCM ainsi que les questions et les réponses associé à chaque question. Chaque QCM respecte le format suivant :

{"mcq1": {"1": {"question": "Lequel des éléments suivants n'est pas un concept POO en Java ?","propositions": [["Compilation", true], ["Héritage", false], ["Polymorphisme", false]]}

Modélisation des classes :

Client

Côté client, la majorité des fonctionnalités sont comprises dans la classe principale de l'application. Le *main*, affiche un menu permettant d'utiliser les différentes fonctionnalités : connexion, demande d'un QCM, demande des scores,… C'est depuis cette classe que les requêtes sont envoyées au serveur.

Une deuxième classe est utilisée, RequestedService, contenant seulement un booléen et une chaîne de caractère, et est utilisée pour bloquer l'affichage du menu et pour sélectionner la gestion de réception d'une réponse du serveur. Elle ne fait rien d'autre en particulier mais sert à contenir les informations nécessaires sur l'ordonnancement des actions.

Serveur

Du côté serveur, la classe principale de l'application permet de recevoir les requêtes envoyés par les clients, et après avoir identifié le service demandé, lance dans un thread correspondant la gestion de la requête. L'utilisation des threads permet de gérer plusieurs requêtes arrivant en même temps : de cette manière, aucun client n'est obligé d'attendre que toutes les autres requêtes soient traitées.

La gestion des utilisateurs est réalisée par la classe ServerManager, qui maintient les sessions actives et permet la vérification de la validité d'un identifiant de session. Dans toute l'application, et donc entre les différents threads utilisés, un seul objet de cette classe est utilisé. La classe en utilise une autre pour représenter un utilisateur, la classe User, qui garde les informations essentielles associées à un client, comme son identifiant de session.

Enfin, la gestion de la base de données s'effectue avec la classe DBManager. Tout accès ou modification des deux fichiers de la base passent par cette classe. Elle comprend également une fonction de génération d'un identifiant de session. Toutes les méthodes sont statiques : aucune instance de cette n'est créé pendant l'exécution du code du serveur.

Tests :

Les tests ont été écrit après avoir fait les classes étant donné que certaines méthodes ont été adaptés au fur et à mesure de l'avancée du projet. Dans les faits, peu de classes ont des tests associés : les classes gérant les fonctionnalités dans des threads, par exemple, n'en ont pas, car leur fonctionnement dépend d'un échange avec un client.

Documentation :

L'utilisation du gestionnaire de QCM s'effectue majoritairement côté client à l'aide du menu affiché par la classe principale. Pour lancer les différents programmes, il faut tout d'abord s'assurer d'avoir l'adresse IP d'un broker MQTT. Ensuite, il suffit de lancer le serveur en ajoutant en arguments en ligne de commande cette adresse IP avec le format suivant : ./graldew app:run --args="tcp://10.11.33.106:1883". Côté client, la démarche est identique. Il est à noter que, pour le client, l'affichage du menu est plus agréable si le paramètre --console=plain est utilisé.

Jeu d'essai :

Un jeu d'essai a été développé et est accessible pour tout utilisateur non connecté dans le code côté client. Ce jeu reproduit les affichages et les possibles entrées données par un utilisateur, et reproduit les actions (dans l'ordre) d'authentification, de demande d'un QCM, de demande des scores, puis de déconnexion. Toutes les requêtes sont réellement envoyées et le code utilise les mêmes fonctions qui auraient été utilisées normalement. Voici ce qui est affiché dans la console du client :

```
> Task :app:run
Que faire ? 1: Authentification, 2: Register, 3: Test
3
Authentification :
Entrer nickname : user_test
Entrer password : 1234
Authentification et/ou Registeration réussie. ID de session :
nQNGYl7mLsOfPbziBcGFoQXZxbeweeKY
Entrer l'ID du QCM (ex: mcq1) : mcq1Question 1 : Lequel des éléments suivants n'est pas un
concept POO en Java ?
0. Compilation ; 1. Héritage ; 2. Polymorphisme
Choix : 1

Question 2 : Quand la surcharge de méthode est-elle déterminée ?
0. Au moment de l'exécution ; 1. Au moment de la compilation ; 2. Au moment du codage
Choix : 2

Question 3 : Quel mot clé permet de rendre une classe accessible à l'extérieur de son package
?
0. global ; 1. public ; 2. super
Choix : 1

Question 4 : Le mot-clé "transient" sert à indiquer
0. qu'un objet est éphémère ; 1. qu'une classe n'est pas instanciable ; 2. qu'un objet n'est pas
serializable
Choix : 0
```

Question 5 : (1==2)?"A":"B"; renvoie
0. A ; 1. B ; 2. AB
Choix : 0

Question 6 : Une interface peut-elle être instanciée ?
0. Oui, si elle est public ; 1. Oui, depuis la version 5.0 ; 2. Non
Choix : 2

Question 7 : Quelle classe fondamentale ne nécessite pas l'instruction "import" car celui-ci est fait automatiquement ?
0. java.lang ; 1. java.main ; 2. java.io
Choix : 1

Question 8 : Qu'est ce qui caractérise le langage Java ?
0. s'exécute à la volée ; 1. la vitesse ; 2. la portabilité
Choix : 2

Question 9 : Le type "float" est codé sur combien d'octets ?
0. 4 ; 1. 6 ; 2. 8
Choix : 2

Question 10 : Quelles méthodes et variables peuvent être utilisées dans une classe héritée ?
0. public ou protected ; 1. private ou public ; 2. toutes
Choix : 2

Question 11 : Quelle méthode permet d'obtenir la liste des méthodes d'une classe ?
0. getClass() ; 1. getMethods() ; 2. ListMethods()
Choix : 1

Question 12 : Le mot clé "extends" est utilisé pour
0. l'héritage d'une classe ; 1. inclure une librairie ; 2. étendre la portée d'une variable
Choix : 2

Question 13 : Si elle n'est pas précisée, la portée d'une variable d'instance dans une classe est
0. locale ; 1. publique ; 2. privée
Choix : 2

Question 14 : Quelle est la différence entre Throw et Throws ?
0. Throw lève une seule exception et Throws plusieurs ; 1. Throws instancie une exception et Throw la déclare ; 2. Throw instancie une exception et Throws la déclare

Choix : 1

Question 15 : On trouve les packages pour gérer les flux dans
0. java.io ; 1. java.ioStream ; 2. java.stream
Choix : 2

Question 16 : Comment connaître la classe mère de Classe1 ?
0. Classe1.getSuperclass() ; 1. Classe1.class.getSuperclass() ; 2. Classe1.getClass()
Choix : 0

Question 17 : Pour empêcher de redéfinir une méthode on la déclare
0. final ; 1. static ; 2. private
Choix : 2

Question 18 : Le mot clé "super" permet
0. de définir une classe prioritaire ; 1. de définir une méthode prioritaire ; 2. d'accéder aux membres d'une classe mère
Choix : 1

Question 19 : Dans la ligne "public class A implements B", B est
0. une extension ; 1. une classe mère ; 2. une interface
Choix : 2

Question 20 : Quel est la différence entre les variables de type "long" et "double" ?
0. long définit des entiers et double des nombres en virgule flottante ; 1. long occupe 2 octets et double 4 octets ; 2. long occupe 4 octets et double 8 octets
Choix : 2

Réponses envoyées au serveur.
Score : 5/20
QCM mcq1 : 18/20
QCM mcq1 : 6/20
QCM mcq1 : 5/20
QCM mcq1 : 8/20
QCM mcq1 : 10/20
QCM mcq1 : 5/20
Disconnection
Déconnexion réussie.

BUILD SUCCESSFUL in 15s

Conclusion :

Le projet permet une gestion de QCM rudimentaire, les fonctions principales ont été implémentées. Cependant, certains points peuvent tout même être améliorés.

Comme mentionné plus tôt, les mots de passe sont saisis en clair dans la console, transmis au serveur et stocké sans chiffrement dans la base de données, ce présente un risque majeur pour la sécurité des informations personnelles.

Également, le projet prévoit une gestion de rôles pour distinguer les étudiants, les professeurs et prévoit même rôle mixte, combinant les droits des deux autres rôles. Pour l'instant, seule la partie étudiante a été implémentée : l'étudiant peut réaliser un QCM et consulter ses scores. Mais aucune fonctionnalité n'a été développer pour ajouter, modifier, et supprimer les QCM de la base de données, d'où le rôle de professeur. Ce-dernier, dans le cas d'un rôle mixte, ne pourrait donc pas effectuer de QCM : là encore l'amélioration est à discuter.